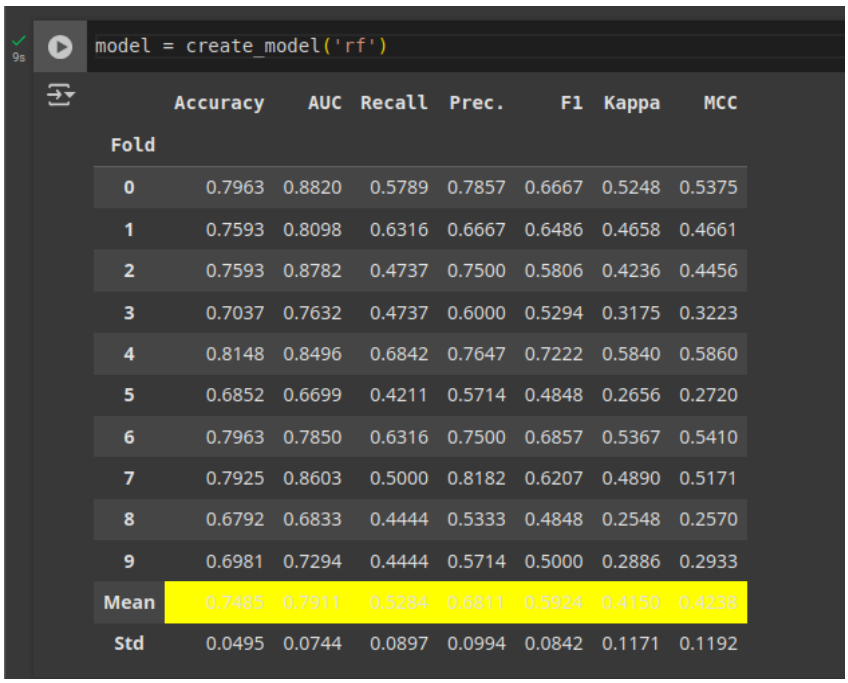
However, if you already know which algorithm you want to use, you can give the following command as shown in Figure 4.

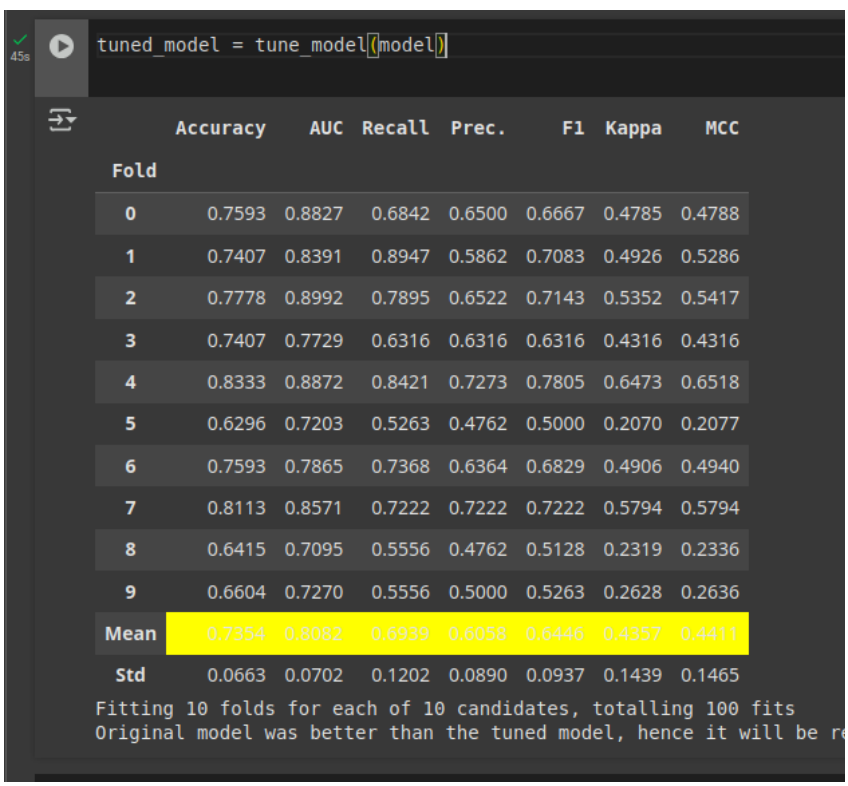
```
model = create_model('rf')
```



```
model = create_model('rf')
```

Fold	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.7963	0.8820	0.5789	0.7857	0.6667	0.5248	0.5375
1	0.7593	0.8098	0.6316	0.6667	0.6486	0.4658	0.4661
2	0.7593	0.8782	0.4737	0.7500	0.5806	0.4236	0.4456
3	0.7037	0.7632	0.4737	0.6000	0.5294	0.3175	0.3223
4	0.8148	0.8496	0.6842	0.7647	0.7222	0.5840	0.5860
5	0.6852	0.6699	0.4211	0.5714	0.4848	0.2656	0.2720
6	0.7963	0.7850	0.6316	0.7500	0.6857	0.5367	0.5410
7	0.7925	0.8603	0.5000	0.8182	0.6207	0.4890	0.5171
8	0.6792	0.6833	0.4444	0.5333	0.4848	0.2548	0.2570
9	0.6981	0.7294	0.4444	0.5714	0.5000	0.2886	0.2933
Mean	0.7485	0.7911	0.5284	0.6811	0.5924	0.4150	0.4238
Std	0.0495	0.0744	0.0897	0.0994	0.0842	0.1171	0.1192

Figure 4: Training on a specific model



```
tuned_model = tune_model(model)
```

Fold	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.7593	0.8827	0.6842	0.6500	0.6667	0.4785	0.4788
1	0.7407	0.8391	0.8947	0.5862	0.7083	0.4926	0.5286
2	0.7778	0.8992	0.7895	0.6522	0.7143	0.5352	0.5417
3	0.7407	0.7729	0.6316	0.6316	0.6316	0.4316	0.4316
4	0.8333	0.8872	0.8421	0.7273	0.7805	0.6473	0.6518
5	0.6296	0.7203	0.5263	0.4762	0.5000	0.2070	0.2077
6	0.7593	0.7865	0.7368	0.6364	0.6829	0.4906	0.4940
7	0.8113	0.8571	0.7222	0.7222	0.7222	0.5794	0.5794
8	0.6415	0.7095	0.5556	0.4762	0.5128	0.2319	0.2336
9	0.6604	0.7270	0.5556	0.5000	0.5263	0.2628	0.2636
Mean	0.7354	0.8082	0.6939	0.6058	0.6446	0.4357	0.4411
Std	0.0663	0.0702	0.1202	0.0890	0.0937	0.1439	0.1465

```
Fitting 10 folds for each of 10 candidates, totalling 100 fits
Original model was better than the tuned model, hence it will be re
```

Figure 5: Fine tuning the model

In this case *rf* is the RandomForest algorithm; you can replace this by *dt* if you want a DecisionTree classifier, and so on.

The next step is to fine tune the model as shown in Figure 5.

```
tuned_model = tune_model(model)
```

Finally, we can evaluate the performance of our model. This can be done using the following line of code as shown in Figure 6.

```
evaluate_model(tuned_model)
```

We can also explore the different parameters of performance visually. We can look at the prediction error and precision recall as shown in Figure 7 and Figure 8 (and more) by clicking on their respective tabs.

To save this model, we can use the following code:

```
save_model(tuned_model, 'diabetes_
prediction')
```

This model is saved as a pickle file that can be used anywhere in any environment, as shown in Figure 9.

We can then load the same model back using the following code:

```
loaded_model = load_model('diabetes_
prediction')
```

As a last step, if we want to predict based on a single instance of data whether a case may turn diabetic or not, we can use the following code, where the parameter values can be modified as needed. The output is shown in Figure 11.

```
import pandas as pd
new_data = pd.DataFrame({
   'Number of times pregnant': [4],
   'Plasma glucose concentration a 2
hours in an oral glucose tolerance test':
[76],
   'Diastolic blood pressure (mm Hg)':
[35],
   'Triceps skin fold thickness (mm)': [0],
```